

PulsePropagation

MMGNLSE Usage Manual

Modal and experimental full-field propagation

Contents

1	Conventions	3
1.1	Power and photon field coordinates	3
2	A scalar single-mode propagation	4
3	Degrees of freedom and domains	6
4	Dispersion	7
4.1	Taylor coefficients	7
4.2	Sampled propagation constants	7
5	Lazy loss and gain arrays	7
6	Overlap tensors and polarization	8
6.1	Spatial overlap	8
6.2	Arbitrary component overlap	8
7	Materials and Raman responses	8
8	Building fiber properties	9
8.1	Analytic fibers	9
8.2	Custom index profiles	10
8.3	Reconstructing the spatial field	11
9	CP compression	11
9.1	Compressing an existing dense overlap	11
9.2	Dense-free overlap construction for sampled modes	11
9.3	What the dense-free error does and does not establish	13
10	Experimental full-field MMGNLSE solver	13
10.1	Model and normalization	14
10.2	Domain, parameters, and initial field	14
10.3	Reference beta, dispersion, and index potential	15
10.4	Integration methods and self-steepening	16
10.5	Saved fields, spectra, and diagnostics	17
10.6	Current full-field restrictions	17
11	CUDA execution	18
11.1	Selecting the backend	18
11.2	Capability matrix	18
11.3	Data movement and returned arrays	19
11.4	Reproducibility and CPU validation	20
11.5	Reusing a CP cache on a GPU worker	21
11.6	RunPod from SSH	21
12	Forward solves and saved planes	23
12.1	Accessing temporal and spectral fields	23
13	Observables	24
14	Adjoint solves	25
15	Reverse-mode autodiff adjoints	26
16	Stochastic ensembles	27

17 Explicit cache files and checksums	28
17.1 Fiber properties	28
17.2 CP decompositions	28
18 Worked examples	29
19 Troubleshooting	30

This manual covers the CPU and optional CUDA dense and CP-compressed modal MMGNLSE interface, the experimental CUDA-only scalar full-field interface, and dense-free construction of modal overlap CP decompositions exposed by PulsePropagation.

1 Conventions

The package uses one set of units throughout the public interface:

Quantity	Unit
time	ps
angular-frequency detuning and ω_0	rad/ps
propagation distance and wave-length	m
modal optical power, $\text{abs2}(A)$	W
full-field intensity, $\text{abs2}(A)$	W/m ²
Taylor coefficient $\beta[k+1]$	ps ^k /m
overlap S	m ⁻²
nonlinear index n_2	m ² /W
loss α and gain	m ⁻¹

Modal A is a power-normalized complex field amplitude, so $\text{abs2}(A)$ is power in watts. The modal overlap remains an inverse area: the user supplies S , n_2 , and ω_0 separately. Full-field normalization is documented below.

1.1 Power and photon field coordinates

Power and photon units are two coordinate systems for the same field. Let $A_{\omega}[k]$ be the centered power-unit spectrum and define

$$w[k] = (Nt * dt) * 1e-12 / (\hbar * \omega_{\text{abs}}[k])$$

for positive absolute angular frequency $\omega_{\text{abs}}[k]$ in rad/s, with dt in ps and $\hbar = 1.054571817e-34$ J s. The weight is zero for a nonpositive absolute frequency. `photon_weights(grid,  $\omega_0$ )` returns these weights.

The forward photon-unit spectrum is

$$B_{\omega}[k] = \sqrt{w[k]} * A_{\omega}[k].$$

Consequently, $\text{abs2}(B_{\omega}[k])$ is the photon number in bin k , and summing it over bins, modes, and polarizations gives the total photon number. The inverse conversion divides by $\sqrt{w[k]}$; bins with zero weight remain zero.

Adjoints are cotangents, so their conversion is dual to the forward-field conversion. If λ_A is in power-field coordinates, then

$$\lambda_B[k] = \lambda_A[k] / \sqrt{w[k]}$$

for a positive-weight bin, and the inverse conversion multiplies by $\sqrt{w[k]}$. This preserves the differential pairing between a field perturbation and its adjoint. It is therefore important not to apply the forward-field scaling to an adjoint.

Both forward and adjoint field accessors default to `units=:power`. `units=:photon` first converts the centered spectral field. A temporal photon-unit field is then defined as

```
Bt = fft(ifftshift(Bomega, 1), 1)
```

It is the Fourier transform of the photon-unit spectrum, not a pointwise rescaling whose squared magnitude is a temporal photon flux.

Public modal fields always retain three axes:

```
time × mode × polarization
```

A saved forward or adjoint solution adds `z` last. A stochastic solution adds trajectory after `z`:

```
time × mode × polarization × z
time × mode × polarization × z × trajectory
```

Suppressed dimensions remain singleton dimensions. Linear polarization means two components ordered `x`, then `y`.

Public full-field arrays use different spatial axes:

```
(time, x, y, polarization)
(time, x, y, polarization, z)
```

The current full-field model is scalar, but its public arrays retain a singleton polarization axis. Its normalization is also spatially resolved: `abs2(A)` is intensity in W/m^2 , and summing over `x` and `y` with the transverse cell area gives instantaneous power in watts.

The public spectral convention is always centered:

```
Aomega = fftshift(ifft(At, 1), 1)
```

Thus frequency samples run from negative to positive detuning, including for odd grid sizes. `frequency_axis(grid)` returns angular-frequency detuning in rad/ps . `wavelength_axis(grid, omega0)` returns the corresponding absolute wavelengths in metres; bins with nonpositive absolute frequency are reported as `Inf`.

2 A scalar single-mode propagation

```
using PulsePropagation

grid = TimeGrid(1024, 10.0)           # 1024 samples over 10 ps
domain = MMGNLSEDomain(
    degrees_of_freedom(:time);
    time_grid=grid,
)

pulse_fwhm = 0.2                       # ps
peak_power = 2.0e3                     # W
```

```

initial = gaussian_pulse(
    domain, 1;
    peak_power=peak_power,
    fwhm=pulse_fwhm,
)

lambda0 = 1550e-9                # m
c = 2.99792458e-4                # m/ps
omega0 = 2pi * c / lambda0       # rad/ps
A_eff = 50e-12                   # m^2

parameters = MMGNLSEParameters(domain;
    length=0.1,                    # m
    alpha=0.0,
    gain=0.0,
    beta=TaylorBeta([0.0, 0.0, -0.0204]),
    S=1 / A_eff,
    n2=2.3e-20,
    omega0=omega0,
    raman=NoRaman(),
)

solution = solve_mmgnlse(initial, parameters, 1e-3; saveat=0.01)

At_final = get_temporal_field(solution)
Aomega_final = get_spectral_field(solution)
z_saved = solution.z

```

The initial field must have exactly the public shape implied by the domain and overlap. Here it is (1024, 1, 1). `gaussian_pulse` interprets `fwhm` as the intensity full width at half maximum and preserves `peak_power` as the total peak power across all launched components.

For a scalar multimode domain created with `degrees_of_freedom(:time, :space)`, provide a mode vector:

```

initial = gaussian_pulse(
    domain, nmodes;
    peak_power=2.0e3,
    fwhm=0.2,
    coefficients=[1.0, 0.5im, zeros(nmodes - 2)...],
)

```

For two linear polarizations, use a `mode × polarization` coefficient matrix. Rows correspond to modes and columns correspond to x and y:

```

coefficients = ComplexF64[
    1.0      0.25im
    0.5im    0.0
    0.0      exp(0.3im)
]

initial = gaussian_pulse(
    polarized_domain, 3;
    peak_power=2.0e3,
    fwhm=0.2,
    coefficients=coefficients,
)

```

)

The accepted coefficient formats are:

Domain	Required coefficients shape
scalar polarization	vector of length <code>nmodes</code>
linear x/y polarization	<code>nmodes × 2</code> matrix

A vector is rejected for a polarized domain, even when only x or y is to be launched; use a matrix with zeros in the other column. Coefficients may be real or complex, must all be finite, and must not all be zero. The helper divides the complete vector or matrix by its Euclidean or Frobenius norm before applying the requested `peak_power`. Thus `peak_power` is the total peak power over all modes and polarizations, while relative complex amplitudes and phases are preserved.

With `coefficients=nothing`, the helper launches one component. `mode=1` and `polarization=1` are the defaults, and polarization indices one and two mean x and y. The `mode` and `polarization` keywords have no effect when explicit coefficients are supplied. Optional `time_offset` and `omega_offset` values are expressed in ps and rad/ps, respectively.

3 Degrees of freedom and domains

Create a nonempty selection from `:time`, `:space`, and `:polarization`:

```
dofs = degrees_of_freedom(:time, :space, :polarization)
domain = MMGNLSEDomain(dofs; time_grid=TimeGrid(2048, 20.0))
```

Names are canonicalized to time, space, polarization order. Calling `degrees_of_freedom()` with no arguments is an error.

Use only the dimensions the problem needs:

```
spatial_domain = MMGNLSEDomain(degrees_of_freedom(:space))
polarized_domain = MMGNLSEDomain(degrees_of_freedom(:polarization))
coupled_domain = MMGNLSEDomain(degrees_of_freedom(:space, :polarization))
```

When `:time` is active, a `TimeGrid` is required. When it is inactive, a grid must not be supplied, the retained time axis has length one, dispersion must be zero, and Raman must be disabled with `NoRaman()`.

Modal domains use `:space` to mean a modal axis and do not require transverse coordinates. A full-field domain instead attaches a uniform Cartesian grid:

```
time_grid = TimeGrid(1024, 10.0) # ps
transverse_grid = TransverseGrid(
    Nx=128,
    Ny=128,
    dx=0.5e-6, # m
    dy=0.5e-6,
)

dofs = degrees_of_freedom(:time, :space)
full_field_domain = MMGNLSEDomain(
```

```
dofs,
time_grid;
transverse_grid=transverse_grid,
)
```

TransverseGrid(x, y) accepts explicit coordinate vectors instead. Both axes must be finite, strictly increasing, and uniformly sampled in metres. The full-field parameter constructor rejects :polarization until vectorial full-field propagation is implemented.

4 Dispersion

4.1 Taylor coefficients

TaylorBeta stores coefficients about the carrier:

```
beta = TaylorBeta([beta0, beta1, beta2, beta3])
```

Its axes are coefficient order, mode, polarization. Omitted trailing axes are broadcast. For example:

```
coefficients = zeros(6, 3, 2) # beta0 through beta5, 3 modes, x/y
beta = TaylorBeta(coefficients)
@assert beta_order(beta) == 5
```

Propagation uses a moving frame: the beta0 and beta1 values of mode 1, polarization 1 define the common carrier phase and group-delay reference. Relative phase and walk-off of every other component are preserved.

4.2 Sampled propagation constants

Use SampledBeta for values sampled directly on the centered detuning grid:

```
omega = frequency_axis(grid) # rad/ps, negative to positive
values = beta_of_omega(omega) # or frequency × mode × polarization
beta = SampledBeta(omega, values)
```

omega must be strictly increasing and must cover every solver detuning. values uses frequency, mode, polarization order and must contain exactly the grid's number of frequency samples.

5 Lazy loss and gain arrays

Loss and gain are power coefficients. Their field-amplitude contribution is

```
(gain - alpha) / 2.
```

Both accept a scalar or a positional array with axes

```
frequency × mode × polarization × z.
```

Rules for omitted dimensions are:

- A scalar broadcasts everywhere.
- A vector always means frequency dependence.

- An array occupies the axes shown above in order.
- Singleton dimensions explicitly skip an axis.
- Multiple z samples are uniformly placed over $[0, \text{length}]$ and linearly interpolated.

Examples:

```
alpha_frequency = collect(range(0.01, 0.03; length=grid.nt))
alpha_mode = reshape([0.01, 0.02, 0.04], 1, 3)
gain_polarization = reshape([0.1, 0.08], 1, 1, 2)
gain_z = reshape([0.0, 0.2, 0.0], 1, 1, 1, 3)
```

Every supplied axis must either be singleton or match its target size, except the sampled z axis, whose length selects the number of uniform samples.

6 Overlap tensors and polarization

For a scalar single-mode problem, $S=1/A_{\text{eff}}$ is sufficient. A raw scalar or four-dimensional array passed to `MMGNLSEParameters` is converted to a `SpatialOverlap`.

6.1 Spatial overlap

```
overlap = SpatialOverlap(Smmmm) # Nm × Nm × Nm × Nm
```

`SpatialOverlap` stores only the spatial tensor. With two polarizations, the silica x/y tensor structure is applied analytically for both instantaneous and delayed nonlinearities. A one-polarization problem is its linear-x reduction.

6.2 Arbitrary component overlap

Use `ComponentOverlap` for a complete space-polarization tensor:

```
overlap = ComponentOverlap(Sfull)
```

`Sfull` has shape

```
Nm × 2 × Nm × 2 × Nm × 2 × Nm × 2.
```

No extra instantaneous polarization coefficients are applied. For delayed Raman, the supplied full component tensor defines the component couplings and the combined material response $h_a + h_b$ filters each $A3 \cdot \text{conj}(A4)$ beating term. Dense and component-CP forward and adjoint paths use the same rule.

7 Materials and Raman responses

Silica supplies Sellmeier dispersion, n_2 , and an anisotropic Raman response by default:

```
material = Silica()
material = Silica(n2=2.3e-20, raman=NoRaman())
material = Silica(raman=AgarwalRaman())
```

Available responses are:

```

NoRaman()

AgarwalRaman(
    fraction=0.18,
    tau1=0.0122,
    tau2=0.0320,
)

AnisotropicRaman(
    fraction=0.245,
    fa=0.75,
    fb=0.21,
    fc=0.04,
    tau1=0.0122,
    tau2=0.0320,
    taub=0.0960,
)

```

All time constants are in ps. Temperature is not part of these material responses. To inspect constructed causal kernels without supplying a sampled kernel yourself:

```

response = raman_response(AnisotropicRaman(), collect(0.0:0.001:0.5))
ha, hb = response.ha, response.hb

```

The Raman fraction is stored separately and is not folded into ha or hb.

8 Building fiber properties

8.1 Analytic fibers

```

grin = GRIN(
    lambda0=1550e-9,
    core_radius=31.25e-6,
    NA=0.275,
)

step = StepIndex(
    lambda0=1550e-9,
    core_radius=12.5e-6,
    NA=0.12,
)

```

The material index at every stencil wavelength comes from material dispersion. NA is held constant across the stencil. GRIN uses the standard parabolic n^2 profile; StepIndex uses weak-guidance scalar LP modes.

Compute modes, dispersion, and overlaps with:

```

properties = compute_fiber_properties(
    grin,
    Silica(),
    15;
    polarization=:linear,

```

```

    mode_basis=:HG,
    beta_order=5,
)

```

GRIN supports `mode_basis=:HG` and `:LP`. Step-index fibers require `:LP`. `polarization=:scalar` retains one polarization; `:linear` retains x/y while sharing the scalar spatial modes and overlap. `beta_order=5` is the default.

Useful accessors are:

```

xy = mode_grid(properties)           # (x=..., y=...)
modes = mode_fields(properties)      # y × x × mode
beta0 = propagation_constants(properties)
S = spatial_overlap(properties)

```

Mode fields satisfy discrete unit-power normalization. Construct propagation parameters directly from the result:

```

parameters = MMGNLSEParameters(domain;
    length=1.0,
    alpha=0.0,
    gain=0.0,
    beta=properties.beta,
    S=properties.S,
    n2=properties.n2,
    omega0=properties.omega0,
    raman=properties.raman,
)

```

8.2 Custom index profiles

A sampled custom profile uses a uniform Cartesian grid and stores its absolute index at `lambda0`:

```

profile = IndexProfile(x, y, nxy; lambda0=1030e-9)

```

`nxy` must have shape `(length(y), length(x))`. Away from `lambda0`, this form preserves its permittivity offset from material dispersion.

For explicit wavelength dependence, supply `n(x, y, lambda)` with all three arguments in metres:

```

index_model = (x, y, lambda) -> begin
    ncore = silica_refractive_index(lambda)
    x^2 + y^2 < (10e-6)^2 ? ncore : sqrt(ncore^2 - 0.12^2)
end

profile = IndexProfile(index_model; lambda0=1030e-9, x=x, y=y)
properties = compute_fiber_properties(
    profile, Silica(), 4;
    mode_basis=:FD,
    beta_order=5,
)

```

Custom modes are computed by the two-dimensional finite-difference Arpack solver. `evaluate_index_profile(profile, material, lambda)` evaluates any profile on its grid.

8.3 Reconstructing the spatial field

```
Axy = get_spatiotemporal_field(solution, properties)
Axy_omega = get_spatiotemporal_field(
    solution, properties; domain=:frequency)
```

The output axes are time/frequency, x, y, polarization. Stochastic output adds trajectory last and accepts `trajectory=:all` or a trajectory index. The reconstruction uses the carrier mode profiles stored in `properties`.

9 CP compression

9.1 Compressing an existing dense overlap

Compress a spatial overlap with a deterministic complex CP decomposition:

```
S_cp = cp_compress(
    properties.S;
    error=1e-4,
    max_rank=1024,
    seed=20260722,
    checksum_fn=canonical_array_checksum,
    fftw_threads=1,
    blas_threads=1,
    npolarizations=2,
)
```

Passing `S_cp` as `S` in `MMGNLSEParameters` selects the compressed contraction automatically. The requested FFTW and BLAS thread counts are applied during compression and the previous process settings are restored afterward.

The fit uses complex factors and a nonconjugated reconstruction. With a fixed seed and settings it is deterministic. If the target error is not reached by `max_rank`, compression throws an error. A zero source tensor produces a rank-zero decomposition and propagation routes through the linear path.

Inspect or validate a result with:

```
rank = cp_rank(S_cp)
iszero_tensor = cp_iszero(S_cp)
S_approx = cp_reconstruct_mmgnlse(S_cp)
validate_cp_source(S_cp, properties.S.values;
    checksum_fn=canonical_array_checksum)
```

Compressing `ComponentOverlap` flattens the full space-polarization tensor and records a component layout automatically.

9.2 Dense-free overlap construction for sampled modes

For many modes, constructing the dense $N_m \times N_m \times N_m \times N_m$ spatial tensor can be more expensive than its eventual CP representation. The dense-free route starts from real scalar mode samples with axes (y, x, mode) on a uniform rectangular grid:

```
source = mode_overlap_quadrature_cp(
    properties.modes;
```

```
    quadrature_order=48,
  )
```

The desired real-scalar overlap is

```
S[p,l,m,n] = integral phi[p] phi[l] phi[m] phi[n] dx dy.
```

The sampled modes are evaluated at a tensor-product Gauss–Legendre grid using local bicubic interpolation. With $q = \text{quadrature_order}$ points in each transverse direction, the integral is represented directly as

```
S[p,l,m,n] ~= sum(r=1:q^2) w[r] U[p,r] U[l,r] U[m,r] U[n,r].
```

Thus the uncompressed source rank is at most q^2 ; q is neither the number of modes nor the final compressed rank. No Nm^4 array is allocated. The same routine accepts explicit fields, x , y arguments:

```
source = mode_overlap_quadrature_cp(
    fields, x, y;
    quadrature_order=48,
)
```

`compute_fiber_properties` can select this representation directly for GRIN, step-index, and finite-difference IndexProfile fibers:

```
properties = compute_fiber_properties(
    fiber,
    material,
    num_modes;
    overlap_representation=:quadrature_cp,
    overlap_quadrature_order=48,
)

@assert properties.S isa MMGNLSECPDecomposition
```

This option returns the quadrature CP source, not a rank-optimized fit. To construct and validate a compressed solver tensor in one call, use:

```
S_cp = cp_compress_mode_overlap(
    properties.modes;
    target_error=1e-4,
    quadrature_order=48,
    authority_order=64,
    quadrature_tolerance=1e-5,
    max_rank=1024,
    seed=20260812,
    backend=:cuda,           # :cpu is also supported
    device=nothing,
    synchronize=true,
)
```

The raw quadrature source reports zero *fit* error because it exactly stores the selected quadrature sum; that value is not a claim of zero quadrature or mode-grid error. `spatial_overlap(properties)` returns either the dense array or CP object appropriate to the selected representation. Load `CUDA.jl` before using `backend=:cuda` for compression.

The routine performs the following operations without reconstructing a dense four-index tensor:

1. Build a working CP quadrature at order `quadrature_order`.
2. Build an authority CP quadrature at the higher `authority_order`.
3. Compute their relative Frobenius difference by blocked CP inner products.
4. Refuse compression if that difference exceeds `quadrature_tolerance`.
5. Compress the working source with the remaining error budget.
6. Verify the final result directly against the authority and require its relative error to be no larger than `target_error`.

For CP tensors A and B, the comparison uses

$$\text{norm}(A-B)^2 = \text{inner}(A,A) + \text{inner}(B,B) - 2 \text{ real}(\text{inner}(A,B)),$$

where every inner product is evaluated from CP factors in bounded blocks. The returned `relative_error` is the final error against the authority, and a deterministic checksum records the sampled mode grid, fields, and authority order. Use `target_error`, rather than the `error` alias accepted by the more general `cp_compress` routine.

The generic dense-free implementation currently requires real scalar sampled modes. Complex modes and general vectorial/component overlaps require an explicit conjugation and component convention and are rejected for now.

9.3 What the dense-free error does and does not establish

The authority comparison establishes convergence with respect to the two chosen *global quadrature orders on the same bicubic interpolant*. It does not by itself establish either of the following:

- that the original rectangular mode grid resolves the continuous fiber eigenmodes;
- that two finite global quadrature orders cannot exhibit false convergence for an under-resolved integrand.

Consequently, treat `relative_error` as a compression-plus-authority-order error, not yet as a rigorous error against the continuous physical overlap. For production validation, increase both quadrature orders, repeat on a refined transverse mode grid, align the modal bases, and compare overlap actions or CP tensors. A future stronger authority can integrate the current piecewise-bicubic interpolant cell by cell: because a product of four bicubic modes has degree at most 12 in each coordinate inside one cell, a 7×7 Gauss–Legendre rule per cell integrates that interpolant to floating-point accuracy.

For analytic GRIN Hermite–Gaussian modes, `grin_hg_overlap_quadrature_cp` provides a stronger special case. If the largest total Hermite order is g , Gauss–Hermite order $2g+1$ integrates all four-mode products exactly up to floating-point error. The routine checks this minimum order before constructing the dense-free source.

10 Experimental full-field MMGNLSE solver

`solve_mmgnlse_full_field` advances a scalar Cartesian (3+1)D field directly on `(time, x, y)` rather than expanding the transverse field in fiber modes. It is experimental and currently CUDA-only. The public API deliberately parallels `solve_mmgnlse(initial_field, parameters, dz; ...)`, but uses `FullFieldMMGNLSEParameters` because diffraction and an index potential replace modal propagation constants and overlap tensors.

10.1 Model and normalization

The diagonal linear generator contains temporal dispersion, power loss or gain, and paraxial diffraction:

```
L(omega,kx,ky) = i D(omega)
                + (gain(omega)-alpha(omega))/2
                - i (kx^2+ky^2)/(2 beta_ref).
```

The carrier-frequency index potential $V(x,y)$ and nonlinear polarization are the residual terms. The full-field amplitude satisfies

```
abs2(A(t,x,y))          units W/m^2
sum(abs2(A), x, y) * dx * dy  units W.
```

Without self-steepening the instantaneous nonlinear coefficient is $i\omega_0 n_2/c$. With self-steepening, the temporal Fourier transform of the instantaneous-plus-delayed nonlinear polarization is instead multiplied by $i(\omega_0 + \omega) n_2/c$ and the spectral-edge damping window described below.

The transverse and temporal grids are Fourier grids, so their numerical boundaries are periodic. Choose windows large enough that the field and any dispersive radiation are negligible at the edges.

10.2 Domain, parameters, and initial field

A complete scalar setup is:

```
using PulsePropagation

time_grid = TimeGrid(1024, 10.0)      # samples, ps window
transverse = TransverseGrid(
    Nx=128, Ny=128,
    dx=0.5e-6, dy=0.5e-6,             # m
)
domain = MMGNLSEDomain(
    degrees_of_freedom(:time, :space),
    time_grid;
    transverse_grid=transverse,
)

c = 2.99792458e-4                      # m/ps
lambda0 = 1550e-9                      # m
omega0 = 2pi * c / lambda0             # rad/ps
nref = 1.444
beta0 = nref * omega0 / c               # m^-1
beta2 = -0.0217                        # ps^2/m, illustrative
beta = TaylorBeta([beta0, 0.0, beta2])

# Index profiles use the fiber convention (y, x).
core_radius = 12.5e-6
ncore = 1.450
index_yx = [
    hypot(x, y) <= core_radius ? ncore : nref
    for y in transverse.y, x in transverse.x
]
potential = paraxial_index_potential(
    domain,
    index_yx;
```

```

    beta=beta,
    omega0=omega0,
)

parameters = FullFieldMMGNLSEParameters(
    domain;
    length=0.01,                # m
    alpha=0.0,                  # m-1 power rate
    gain=0.0,                   # m-1 power rate
    beta=beta,
    potential=potential,        # axes (x, y), m-1
    n2=2.3e-20,                 # m2/W
    omega0=omega0,
    raman=NoRaman(),
    self_steepening=true,
)

t = time_axis(time_grid)
temporal = exp.(-2log(2) .* (t ./ 0.2).^2)
spatial = [
    exp(-((x / 5e-6)^2 + (y / 5e-6)^2))
    for x in transverse.x, y in transverse.y
]
peak_intensity = 1e14          # W/m2
initial_scalar = sqrt(peak_intensity) .*
    reshape(temporal, :, 1, 1) .*
    reshape(spatial, 1, size(spatial)...)
initial_field = reshape(
    initial_scalar,
    full_field_shape(parameters),
)

```

The public initial field must be an ordinary or device-compatible array with exact axes (time, x, y, polarization) and a singleton fourth axis. Ordinary Julia arrays are transferred to the selected GPU. A domain containing :polarization, a three-dimensional input that omits the singleton axis, or a nonuniform transverse grid is rejected explicitly.

alpha and gain may be nonnegative scalars or centered frequency vectors of length N_t ; they are currently independent of z, x, and y. potential may be a real scalar or an (x, y) matrix in m^{-1} . NoRaman(), AgarwalRaman(), and AnisotropicRaman() are accepted; the scalar full-field model uses the combined scalar delayed response.

10.3 Reference beta, dispersion, and index potential

beta may be TaylorBeta or SampledBeta, but it must have singleton mode and polarization axes. As in the modal solver, the temporal operator removes the common $\beta_0 + \beta_1\omega$ reference line. Consequently, passing

```
TaylorBeta([beta0, beta1, beta2, beta3, ...])
```

uses beta2 and all higher supplied coefficients for temporal dispersion. The positive carrier beta0 also defines the diffraction reference β_{ref} ; the associated homogeneous reference index is

```
n_ref = beta_ref * c / omega0.
```

If `beta` contains only relative dispersion and therefore has zero `beta0`, supply `reference_index` explicitly. If both a positive `beta0` and `reference_index` are supplied, they must agree within the constructor's tolerance.

`paraxial_index_potential` converts an index array with fiber axes (`y`, `x`) to the full-field (`x`, `y`) convention using

$$V(x,y) = k_0^2 [n(x,y)^2 - n_{\text{ref}}^2] / (2 \beta_{\text{ref}}),$$

$$k_0 = \omega_0/c.$$

The potential is evaluated at the carrier and is frequency independent in the current paraxial model. A constant offset changes only the common phase, but a consistent reference is important when comparing against modal propagation constants.

10.4 Integration methods and self-steepening

Run the fourth-order method with:

```
solution = solve_mmgnlse_full_field(
    initial_field,
    parameters,
    1e-5;
    method=RK4IP(),
    precision=:float64,
    saveat=nothing,
    backend=:cuda,
    device=nothing,
    synchronize=true,
)
```

`RK4IP()` is the default. The solver stores its advancing state in joint (`omega`, `kx`, `ky`) space, applies diffraction, temporal dispersion, and spectral loss or gain through the exact diagonal propagator, and evaluates the index potential plus nonlinear polarization as a four-stage interaction-picture Runge–Kutta residual. Each nonlinear stage is transformed to (`time`, `x`, `y`) as needed. The implementation reuses a field, stage, derivative, accumulator, and optional mixed-representation workspace instead of retaining four complete modal-style stage arrays.

Because the potential is explicit in `RK4IP`, the solver checks the necessary pure-potential stability condition before GPU allocation:

```
min(dz, length) * maximum(abs, potential) <= 2sqrt(2).
```

This is not a sufficient nonlinear accuracy condition; reduce `dz` further until the desired observable converges.

`StrangSplit()` selects the second-order symmetric method:

```
reference = solve_mmgnlse_full_field(
    initial_field, parameters, dz;
    method=StrangSplit(),
    precision=:float64,
)
```

It applies exact diagonal half-steps around the real-space residual. Without self-steepening, the local potential/Kerr/Raman phase subflow is applied pointwise. With self-steepening, potential half-phases surround an explicit midpoint update of the nonlinear shock term, retaining second-order accuracy.

With `self_steepening=true`, the nonlinear polarization is multiplied in frequency space by the frequency-dependent factor proportional to $\omega_0 + \omega$, together with the solver's spectral-edge damping window. The index potential is not multiplied by this shock prefactor. The public full-field default is `true`; this flag has intentionally not been added to the modal `MMGNLSEParameters` interface.

For a fixed spatial and temporal grid, a convergence study should exhibit second-order Δz error for `StrangSplit()` and fourth-order Δz error for `RK4IP()` before floating-point, FFT-grid, or model error dominates. Agreement between the two methods at successively smaller Δz is the recommended first cross-check.

10.5 Saved fields, spectra, and diagnostics

Full-field `saveat` follows the modal solver rules: nothing saves endpoints, `:steps` saves every landed step, a scalar requests uniform spacing, and a vector requests explicit positions. The endpoint is always retained. Returned fields are host arrays with axes (time, x, y, polarization, z).

```
At = get_temporal_field(solution; z=:final)
Aomega = get_spectral_field(solution; z=:all)
Axy_t = get_spatial_field(solution; time=0.0, z=:final)
Axy_lambda = get_spatial_field(
    solution; wavelength=1550e-9, z=:final)
power_t = full_field_power(solution; z=:final)
```

`get_spectral_field` uses the same centered spectral convention as the modal API. `get_spatial_field` selects the nearest time or absolute wavelength and returns (x, y); exactly one selector is required. `full_field_power` accepts one saved plane and integrates $\text{abs}^2(A)$ over the transverse grid. Full-field accessors do not currently expose the modal API's `units=:photon` conversion.

The lower-level experimental helpers `reconstruct_full_field`, `project_full_field`, `full_field_subspace_leakage`, and `phase_invariant_relative_error` support comparisons with a sampled modal basis. Fiber mode arrays use (y, x, mode), while the full-field solver uses (x, y), and the helpers perform that axis conversion.

The solution cache records the selected GPU, precision, accepted steps, workspace and field byte estimates, and counts of joint, temporal, spatial, Raman, and self-steepening FFT operations. These counters are intended for profiling and regression checks rather than as a stable performance contract.

`FullField3DGrid`, `FullField3DParameters`, and `solve_full_field_3d` remain available as the lower-level experimental benchmarking interface. They use a three-axis scalar input and symbol-valued method selection. New application code should prefer `MMGNLSEDomain`, `FullFieldMMGNLSEParameters`, and `solve_mmgnlse_full_field` so field axes and solver selection follow the public `MMGNLSE` conventions.

10.6 Current full-field restrictions

- CUDA execution only; there is no CPU full-field backend.
- Scalar polarization only, with a retained singleton public axis.
- Fixed-step `RK4IP()` and `StrangSplit()` only; no adaptive controller.
- No full-field adjoint, stochastic ensemble, or reverse-mode autodiff path.

- Uniform Cartesian Fourier grids and periodic numerical boundaries.
- A carrier-frequency, longitudinally invariant real index potential.
- Longitudinally invariant scalar or frequency-dependent loss and gain.
- Host-returned saved fields; saving many planes can dominate host memory and transfer time.

11 CUDA execution

CUDA is an optional dependency. Install it in the active project, then load CUDA once in the Julia process so the package extension is activated:

```
julia --project=. -e 'using Pkg; Pkg.add("CUDA"); Pkg.precompile()'
julia --project=. -e 'using CUDA; CUDA.versioninfo(); @assert CUDA.functional()'
```

The project accepts CUDA.jl major versions 5 and 6. A functional NVIDIA driver and a GPU assigned to the process are also required. Loading PulsePropagation without CUDA installed remains a CPU-only configuration.

11.1 Selecting the backend

The public solvers use symbols rather than exposing backend implementation types:

```
forward = solve_mmgnlse(
    initial,
    parameters,
    dz;
    method=RK4IP(),
    saveat=:steps,
    backend=:cuda,
    device=nothing,
    synchronize=true,
)
```

The same backend, device, and synchronize keywords are accepted by `solve_adjoint` and `solve_mmgnlse_stochastic`. The default is `backend=:cpu` for the modal solvers. Use `backend=:cuda` for modal CUDA execution. `solve_mmgnlse_full_field` is CUDA-only and defaults to `backend=:cuda`.

`device=nothing` uses CUDA.jl's current device. An integer selects a device; for example, `device=0` selects the first visible GPU. Device indices refer to the process-visible devices after settings such as `CUDA_VISIBLE_DEVICES` have been applied.

Keep `synchronize=true` while validating or timing an end-to-end solve. Setting it to `false` skips the facade's explicit barrier. Required host copies still wait for their device work, so a returned solution is never a lazy or incomplete GPU result.

11.2 Capability matrix

The CUDA facade is designed to preserve the same physical parameters and array conventions as the CPU facade. "Regression-covered" below means the optional CUDA suite exercises the route when a functional device is available; on a machine without one, those tests are skipped.

Capability	backend=:cpu	backend=:cuda
Fixed-step RK4IP() forward solve and exact saves	yes	regression-covered
Fixed-step Tsit5() forward solve	yes	regression-covered
Dense SpatialOverlap, forward and adjoint	yes	regression-covered
Spatial CP decomposition, forward and adjoint	yes	regression-covered
Dense ComponentOverlap and component CP, forward and adjoint	yes	regression-covered
Scalar and linear x/y polarization, forward and adjoint	yes	regression-covered
Scalar and mode/polarization alpha and gain broadcasting	yes	regression-covered
Fully frequency-, mode-, polarization-, and z-varying alpha and gain	yes	regression-covered
TaylorBeta	yes	regression-covered
SampledBeta	yes	regression-covered
NoRaman, AgarwalRaman, and AnisotropicRaman, forward and adjoint	yes	regression-covered
Adjoint loss/gain signs and units=:power/:photon	yes	regression-covered
Default interaction-picture adjoint invocation	yes	regression-covered
Seeded input-noise stochastic ensemble	yes	regression-covered
Reverse-mode Zygote autodiff adjoint	checkpointed	no
Fiber-mode construction	host	host
Dense or factorized cp_compress fitting	yes	yes
Generic cp_compress_mode_overlap fitting	yes	yes
Scalar full-field RK4IP() and StrangSplit()	no	experimental
Full-field polarization, adjoint, and stochastic solves	no	no

CUDA forward propagation supports fixed-step `method=RK4IP()` and fixed-step `method=Tsit5()`. For a CUDA adjoint, use the public default `Vern9()` method selection rather than passing an alternative integration method. The returned adjoint still follows the public centered-spectral terminal convention and has ascending saved `z` coordinates.

Fiber-mode construction, JLD2/TOML cache validation, observable evaluation, and result serialization run on the host. CP fitting can use either `backend=:cpu` or `backend=:cuda`; source modes and returned cache data remain ordinary host arrays. A verified CP decomposition can therefore be prepared once and reused by CUDA modal solves.

11.3 Data movement and returned arrays

Pass ordinary Julia arrays and the same modal `MMGNLSEParameters` used for CPU execution, or use `FullFieldMMGNLSEParameters` with the full-field solver. With `backend=:cuda`, active fields, operators, overlap factors, and solver workspaces are copied to and retained on the selected GPU during the numerical

solve. Public solution data is copied back before return:

```
@assert forward.fields isa Array
@assert get_spectral_field(forward) isa Array
```

Forward, adjoint, and stochastic results therefore have exactly the host array axes documented above. Do not wrap the public initial field or overlap in a device-array type. Saved trajectories can be the dominant host transfer and memory cost; use `saveat=nothing` when only endpoints are needed and `saveat=:steps` when the adjoint or a longitudinal contraction requires every landed plane.

11.4 Reproducibility and CPU validation

Use explicit seeds for both CP fitting and stochastic input noise:

```
S_cp = cp_compress(
    properties.S;
    error=1e-3,
    seed=20260724,
    checksum_fn=canonical_array_checksum,
)

ensemble = solve_mmgnlse_stochastic(
    initial, parameters, dz, 100;
    noise=1.0,
    seed=13792,
    backend=:cuda,
)
```

The stochastic seed fixes the host-generated input perturbations, including independent polarization samples. It does not make floating-point reductions on different GPU models bitwise identical. Compare CPU and GPU calculations with tolerances and physics-level observables, not byte equality.

For a new parameter regime, first run a reduced problem on both backends:

```
cpu = solve_mmgnlse(
    initial, parameters, dz;
    method=RK4IP(), saveat=nothing, backend=:cpu)
gpu = solve_mmgnlse(
    initial, parameters, dz;
    method=RK4IP(), saveat=nothing, backend=:cuda,
    synchronize=true)

relative_l2 = norm(
    get_spectral_field(gpu) .- get_spectral_field(cpu)) /
    max(norm(get_spectral_field(cpu)), eps(Float64))
@assert relative_l2 <= 2e-7
```

The $2e-7$ forward threshold is a practical smoke-test value for the reduced worked example, not a universal error guarantee. Its example adjoint uses $2e-5$. Tighten or relax tolerances only after recording the grid, step size, precision, GPU model, CUDA versions, CP checksum, and the corresponding CPU convergence study.

For adjoint validation, form `lambda_L` from the same observable on each backend's forward result, compare the input-plane spectral adjoints, and also check a centered finite-difference directional derivative.

Save the forward with `saveat=:steps` during this check. `solve_adjoint` returns its trajectory in ascending `z` order even though it integrates from the output plane toward the input.

11.5 Reusing a CP cache on a GPU worker

CP caches are portable host files and retain the source-tensor checksum:

```
S_dense = properties.S.values
S_cp = load_cp_decomposition(
    "/workspace/cache/step_index_cp.jld2",
    "/workspace/cache/step_index_cp.toml";
    source_S=S_dense,
)

parameters = MMGNLSEParameters(domain;
    length=length_m,
    beta=properties.beta,
    S=S_cp,
    n2=properties.n2,
    omega0=properties.omega0,
    raman=properties.raman,
)
```

Keep both cache files together. A partial pair, file corruption, or a different dense overlap is rejected before GPU work begins.

11.6 RunPod from SSH

Choose a CUDA-capable pod, expose SSH, and attach a persistent volume at `/workspace` if caches and results must survive pod termination. After connecting:

```
nvidia-smi

# Install Julia when the pod image does not already provide it.
curl -fsSL https://install.julialang.org | sh -s -- -y
export PATH="$HOME/.juliaup/bin:$PATH"
juliaup add 1.12
juliaup default 1.12

cd /workspace
git clone https://github.com/nrivera494/FastDifferentiableMMNLO.git
cd FastDifferentiableMMNLO

julia --project=. -e 'using Pkg; Pkg.instantiate(); Pkg.add("CUDA"); Pkg.precompile()'
julia --project=. -e 'using CUDA; CUDA.versioninfo(); @assert CUDA.functional()'
julia --project=. test/test_mmgnlse_cuda_optional.jl
```

Run the reduced workflow first. Its default performs CPU/GPU forward and adjoint checks and writes both its CP cache and result file under `/workspace`:

```
mkdir -p /workspace/pulseprop-cache /workspace/pulseprop-results

PULSEPROP_BACKEND=cuda \
PULSEPROP_VALIDATE_CPU=true \
PULSEPROP_CACHE_DIR=/workspace/pulseprop-cache \
```

```
PULSEPROP_OUTPUT_DIR=/workspace/pulseprop-results \
julia --project=. examples/new_api/step_index_squeezing_gpu.jl
```

Then run the notebook-scale grid and propagation length:

```
PULSEPROP_BACKEND=cuda \
PULSEPROP_FULL_EXAMPLES=true \
PULSEPROP_VALIDATE_CPU=false \
PULSEPROP_NTRAJ=100 \
PULSEPROP_CACHE_DIR=/workspace/pulseprop-cache \
PULSEPROP_OUTPUT_DIR=/workspace/pulseprop-results \
julia --project=. examples/new_api/step_index_squeezing_gpu.jl
```

The full branch keeps the notebook's numerical settings: $N_t=2048$, a 10 ps window, three scalar LP modes, 2.86 m length, dense-forward $dz=2.86/4000$ m, CP-forward $dz=1e-3$ m, $saveat=0.1$ m for both forwards, adjoint $dz=2.86/4000$ m, CP target error $1e-3$, and 100 stochastic trajectories. CPU/GPU parity, when enabled, repeats those settings rather than substituting a cheaper reference.

Useful controls are:

Environment variable	Meaning	Default
PULSEPROP_BACKEND	cpu or cuda	cuda
PULSEPROP_FULL_EXAMPLES	notebook-scale settings	false
PULSEPROP_CUDA_DEVICE	visible integer device index	current device
PULSEPROP_CUDA_SYNCHRONIZE	explicit CUDA synchronization	true
PULSEPROP_VALIDATE_CPU	CPU reference checks at the selected resolution	true for reduced CUDA
PULSEPROP_FORWARD_RTOL	reduced forward relative-L2 limit	$2e-7$
PULSEPROP_ADJOINT_RTOL	reduced adjoint relative-L2 limit	$2e-5$
PULSEPROP_CP_SEED	deterministic CP fit seed	20260724
PULSEPROP_NOISE_SEED	deterministic input-noise seed	13792
PULSEPROP_NTRAJ	stochastic trajectory count	8 reduced, 100 full
PULSEPROP_CACHE_DIR	paired CP cache location	repository output cache
PULSEPROP_OUTPUT_DIR	JLD2 result location	repository output

The result JLD2 contains host arrays for the final dense and CP temporal and spectral fields, their relative error, the input adjoint, stochastic filtered-photon samples, saved z coordinates, checksums, seeds, settings, and any CPU/GPU validation errors. Copy that file and both CP cache files off ephemeral storage

before stopping the pod.

12 Forward solves and saved planes

```
solution = solve_mmgnlse(
    initial,
    parameters,
    dz;
    method=RK4IP(),
    saveat=nothing,
)
```

RK4IP() is the default fixed-step method. Tsit5() is also available and is run with adaptivity disabled at the requested dz.

Saving rules are:

- saveat=nothing: save only the beginning and end.
- saveat=:steps: save every landed integration step.
- saveat=spacing: save at that distance interval.
- saveat=[z1, z2, ...]: save at the requested positions.

The exact endpoint is always appended, and steps are shortened as necessary to land on every requested plane. solution.z is the authoritative list of saved coordinates.

12.1 Accessing temporal and spectral fields

Use the accessors rather than relying on the stored representation:

```
At = get_temporal_field(solution; z=:final, units=:power)
Aomega = get_spectral_field(solution; z=:all, units=:photon)
Bt = get_temporal_field(solution; z=:all, units=:photon)

lambda_t = get_temporal_field(adjoint; z=:all, units=:power)
lambda_omega_0 = get_spectral_field(
    adjoint; z=:initial, units=:photon)

At_ensemble = get_temporal_field(
    ensemble; z=:final, trajectory=:all, units=:power)
```

The public defaults are:

Object	Default z	Default units	Other selector
forward solution	:final	:power	none
adjoint solution	:initial	:power	none
stochastic ensemble	:final	:power	trajectory=:all

These defaults also apply to calls without keywords, so existing calls such as get_spectral_field(solution) continue to work and return the final power-unit field. For either accessor, z accepts :initial, :final, :all, a saved-plane integer index, or a real coordinate. A real coordinate selects the

nearest saved plane and does not interpolate. `z=:all` retains the saved-z axis. A stochastic trajectory selector accepts `:all` or one integer trajectory index.

`get_spectral_field` returns the centered negative-to-positive spectrum

```
Aomega = fftshift(ifft(At, 1), 1)
```

with no additional public prefactor. `get_temporal_field` applies the inverse operation after any requested unit conversion. Forward solutions are stored as power-unit temporal fields. An adjoint may be stored in the units requested from `solve_adjoint`, but its accessors still default to `units=:power` and perform the needed conversion. The `units=:photon` definitions, including the dual conversion for adjoints, are given earlier under “Power and photon field coordinates.”

13 Observables

All spectral observables use centered ordering. The most common constructor defaults are:

Constructor	Required argument	Defaults	Meaning with defaults
SpectralPhotonNumber	filter	modes=:all	filtered photon number over all modes and polarizations
FilterEnergy	filter	domain=:frequency, modes=:all	filtered squared-field sum in frequency coordinates
BinEnergy	index	domain=:frequency	squared field at one frequency bin; omitted mode or polarization indices default to one
TemporalMoment	none	order=1, center=0. 0, normalized=true, modes=:all	energy-normalized temporal centroid
SpectralMoment	none	order=1, center=0. 0, normalized=true, modes=:all	energy-normalized mean angular-frequency detuning
TemporalCentroid	none	modes=:all	alias for the default temporal moment
SpectralCentroid	none	modes=:all	alias for the default spectral moment

For `SpectralPhotonNumber`, write `filter=nothing` explicitly to include every spectral bin, or use a scalar filter such as `filter=1.0`. Filters may be a real scalar or a lazy array with axes frequency or time, mode, polarization; singleton dimensions broadcast. A filter weights `abs2(field)` in the objective. For an amplitude transfer function `H`, therefore supply `abs2.(H)` as the filter. `modes` selects spatial modes; all polarizations are retained.

For the variance (squared RMS width), use `order=2`, `center=:centroid`, and `normalized=true`; take `sqrt(value(...))` for the RMS width. In contrast, `order=1`, `center=:centroid` is the first central moment and is zero, not the centroid. Setting `normalized=false` returns the unnormalized field-weighted moment.

Common constructors are:

```

photon_observable = SpectralPhotonNumber(filter=filter, modes=:all)
energy_observable = FilterEnergy(
    filter=filter, domain=:frequency, modes=:all)
bin_observable = BinEnergy(index=(frequency_index, mode, polarization))
temporal_centroid = TemporalCentroid(modes=:all)
spectral_centroid = SpectralCentroid(modes=:all)
temporal_width = TemporalMoment(
    order=2, center=:centroid, normalized=true, modes=:all)
spectral_width = SpectralMoment(
    order=2, center=:centroid, normalized=true, modes=:all)
ratio = RatioObservable(photon_observable, energy_observable)

```

Evaluate an observable and obtain its centered spectral terminal condition:

```

J = value(photon_observable, solution)
lambda_L = terminal_condition(photon_observable, solution)

```

`value(observable, solution)` evaluates the final saved plane by default. `value(observable, ensemble)` returns one value per stochastic trajectory; statistical summaries such as mean and var are ordinary postprocessing. The ensemble overload accepts no `z` keyword and evaluates the final saved plane.

`lambda_L` is the Wirtinger derivative $dJ/d\text{conj}(A_{\text{omega}}(L))$ and can be passed directly to `solve_adjoint`.

Convenience diagnostics include:

```

photons = photon_spectrum(solution)
N = photon_number(solution, filter; modes=:all)
energy_nJ = modal_energy(solution; sum_polarizations=false)
weights = photon_weights(grid, parameters.omega0)

```

`photon_spectrum(...; normalization=:bin)` returns photons per bin; `normalization=:density` returns density per angular-frequency sampling interval. Nonpositive absolute-frequency bins have zero photon weight. Photon-number diagnostics require the `:time` degree of freedom because the bin normalization depends on the temporal window.

14 Adjoint solves

Given a saved forward solution:

```

lambda_L = terminal_condition(observable, forward)

adjoint = solve_adjoint(
    lambda_L,
    parameters,
    dz_adj;
    forward_solution=forward,
    method=Vern9(),
    units=:power,
    saveat=:steps,
)

```

Alternatively, ask the routine to generate its own forward trajectory:

```
adjoint = solve_adjoint(
    lambda_L,
    parameters,
    dz_adj;
    initial_field=initial,
    dz_forward=dz,
)
```

Supply exactly one of `forward_solution` and `initial_field`. When an initial field is supplied, `dz_forward` is required. A sparsely saved forward solution is replayed at its integration steps before the adjoint solve.

The adjoint is integrated in the interaction picture with fixed-step `Vern9()` applied only to the nonlinear VJP. Dispersion and longitudinally varying gain/loss use their exact diagonal propagators, matching the forward RK4IP linear convention. Returned `z` coordinates are ascending even though integration proceeds backward. The input `lambda_L` is always expressed in power-field coordinates. `units=:photon` converts the saved returned gradient to $dJ/d\text{conj}(B)$ and zeros nonphysical bins; `units=:power` retains power-field coordinates.

Dense and CP paths support loss, gain, spatial or complete component tensors, linear x/y polarization, `NoRaman()`, `AgarwalRaman()`, and `AnisotropicRaman()`.

15 Reverse-mode autodiff adjoints

`get_autodiff_adjoint` computes an independent reverse-mode derivative through the discrete forward solver:

```
lambda_omega_0 = get_autodiff_adjoint(
    observable,
    initial,
    parameters,
    dz;
    method=RK4IP(),
    units=:power,
    checkpointing=:auto,
)
```

The function performs its own forward solve, constructs the observable's terminal condition, and differentiates the fixed-step RK4IP propagation with `Zygote`. It returns the centered spectral cotangent at `z=0` with shape (frequency, mode, polarization). It does not return an adjoint solution or a saved trajectory.

This result is the exact reverse derivative of the *discretized* RK4IP propagation. It is intended to agree, as the step size converges, with

```
get_spectral_field(
    solve_adjoint(...);
    z=:initial,
    units=units,
)
```

where `solve_adjoint` integrates the continuous interaction-picture adjoint. The two routes are therefore a useful regression comparison but need not be bitwise equal at finite `dz`. Use `solve_adjoint` when a longitudinal adjoint trajectory is required.

The checkpointing choices are:

checkpointing	Behavior
:auto	blocks of 10 RK4IP steps
positive integer <code>n</code>	at most <code>n</code> RK4IP steps per block
:none	retain the full reverse-mode tape

Checkpointing stores only block boundaries and recomputes each block during the reverse pass. It therefore trades roughly one additional forward trajectory of computation for a much smaller active tape. A smaller block uses less memory and more boundaries; a larger block uses more memory.

The current autodiff route is CPU-only, uses Zygote, and accepts only `method=RK4IP()`. `units=:power` is the default; `units=:photon` applies the dual adjoint conversion described above and zeros nonpositive absolute-frequency bins. CUDA forward and continuous-adjoint solves remain available through `backend=:cuda`, but GPU reverse-mode autodiff is not currently exposed.

16 Stochastic ensembles

```
ensemble = solve_mmgnlse_stochastic(
    initial,
    parameters,
    dz,
    100;
    noise=1.0,
    seed=1234,
    saveat=:steps,
)
```

`noise` is an input photon-number Fano factor:

- A scalar is uniform across frequency, mode, and polarization.
- A vector is frequency-dependent.
- Positional arrays use frequency, mode, polarization order and may use singleton dimensions.

Stochastic photon-noise ensembles require the `:time` degree of freedom.

Each frequency-mode-polarization component is independent and sampled as

```
delta_B = sqrt(F)/2 * (xi1 + i*xi2),
```

where `xi1` and `xi2` are independent unit normal variables. The `x` and `y` noises are independent with the same Fano factor when that axis is broadcast. Supplying a fixed `seed` makes the ensemble deterministic. `seed` and `rng` cannot be supplied together.

Access one or all trajectories with:

```
At_one = get_temporal_field(ensemble; trajectory=1)
Aomega_all = get_spectral_field(ensemble; trajectory=:all)
```

This routine samples initial fluctuations only. It does not add distributed Raman or gain Langevin sources. Consequently, complete QSA squeezing and timing-jitter curves that require longitudinal source contractions need an additional calculation over the saved forward and adjoint z coordinates. There is no temperature keyword for this input-noise sampler; temperature belongs with the correlation model used for a distributed Langevin source.

17 Explicit cache files and checksums

Fiber properties and CP decompositions use paired JLD2 data and TOML metadata files. Paths are explicit, parent directories must already exist, and existing files are protected unless `overwrite=true` is supplied.

17.1 Fiber properties

```
save_fiber_properties(
    "cache/fiber.jld2",
    "cache/fiber.toml",
    properties,
)

checksum = fiber_properties_checksum(properties)

loaded_properties = load_fiber_properties(
    "cache/fiber.jld2",
    "cache/fiber.toml";
    expected_checksum=checksum,
    source_S=properties.S.values,
)
```

`FiberProperties` may contain either a dense `SpatialOverlap` or an `MMGNLSECPDecomposition`. For properties constructed with `overlap_representation=:quadrature_cp`, omit `source_S`; the cache stores and validates the CP weights, all four factors, layout, source checksum, reported error, and convergence metadata:

```
save_fiber_properties(
    "cache/fiber_quadrature_cp.jld2",
    "cache/fiber_quadrature_cp.toml",
    properties,
)

loaded_properties = load_fiber_properties(
    "cache/fiber_quadrature_cp.jld2",
    "cache/fiber_quadrature_cp.toml";
    expected_checksum=fiber_properties_checksum(properties),
)

@assert loaded_properties.S isa MMGNLSECPDecomposition
```

17.2 CP decompositions

Attach source identity during compression or saving:

```

S_dense = properties.S.values
S_cp = cp_compress(
    S_dense;
    error=1e-4,
    max_rank=1024,
    checksum_fn=canonical_array_checksum,
)

save_cp_decomposition(
    "cache/overlap_cp.jld2",
    "cache/overlap_cp.toml",
    S_cp;
    source_S=S_dense,
)

loaded_cp = load_cp_decomposition(
    "cache/overlap_cp.jld2",
    "cache/overlap_cp.toml";
    source_S=S_dense,
)

```

A result from `cp_compress_mode_overlap` already carries a checksum of its sampled modes and authority order, so it can be cached without supplying a dense source:

```

save_cp_decomposition(
    "cache/overlap_dense_free_cp.jld2",
    "cache/overlap_dense_free_cp.toml",
    S_cp,
)

loaded_cp = load_cp_decomposition(
    "cache/overlap_dense_free_cp.jld2",
    "cache/overlap_dense_free_cp.toml",
)

```

Loading validates the JLD2 SHA-256 checksum, individual arrays, logical payload metadata, and any supplied expected/source checksum. A changed source tensor is rejected.

18 Worked examples

Five executable workflows are provided:

- `birefringent_mm_soliton.jl`
- `gif625_analytic.jl`
- `squeezing_spectral_filter.jl`
- `soliton_jitter.jl`
- `step_index_squeezing_gpu.jl`

Focused validation entry points for the features documented here are:

```

julia --project=. benchmarks/validate_full_field_host.jl
julia --project=. benchmarks/validate_generic_mode_cp.jl

# On a CUDA worker after loading/adding CUDA.jl:

```

```
julia --project=. benchmarks/validate_full_field_cuda.jl
julia --project=. benchmarks/validate_generic_mode_cp_cuda.jl
```

The host full-field test checks the public constructors, axes, accessors, and parameter conversion without attempting propagation. The CUDA test compares the public wrapper to the validated low-level kernel and exercises both full-field methods. The generic CP tests check quadrature convergence, dense-free compression, cache round-tripping, and the CUDA compression path.

They use reduced settings by default. Select the notebook-scale grids, pulse parameters, fiber lengths, and step sizes with:

```
PULSEPROP_FULL_EXAMPLES=true julia --project=. \
  examples/new_api/birefringent_mm_soliton.jl
```

The squeezing and jitter workflows exercise the forward, adjoint, filter, and input-noise pieces. Their distributed Langevin-source contribution is not included, as described in the stochastic section.

The step-index squeezing script is the executable reduced/full translation of [step_index_squeezing.ipynb](#). It defaults to CUDA, caches its CP decomposition, uses deterministic input noise, optionally validates against CPU, and writes a portable JLD2 result. See the RunPod section for complete commands.

19 Troubleshooting

Symptom	Check
A field-shape error	Retain time, mode, polarization axes, including singleton dimensions.
A sampled-dispersion error	Use <code>frequency_axis(grid)</code> in rad/ps and supply exactly <code>Nt</code> increasing samples.
A no-time parameter error	Use zero dispersion, <code>NoRaman()</code> , no <code>TimeGrid</code> , and an initial time axis of length one.
A loss/gain axis error	Use frequency, mode, polarization, z order; add singleton dimensions to reach a later axis.
A CP compression failure	Relax error, increase <code>max_rank</code> , or increase <code>maxiter</code> ; missed targets never return silently.
Dense-free quadrature is reported unconverged	Increase <code>quadrature_order</code> and <code>authority_order</code> ; also refine the sampled mode grid before interpreting the result as physical accuracy.
A dense-free CP call rejects the mode fields	The generic routine currently requires real scalar fields with axes <code>(y, x, mode)</code> on finite uniform grids.
A cache checksum error	Load with the same dense source tensor and an unmodified JLD2/TOML pair.
Unexpected spectral order	Public spectra and axes are already centered; do not apply an additional shift.

Continued on next page

Symptom	Check
Zygote autodiff exhausts memory	Use <code>get_autodiff_adjoint(...; checkpointing=:auto)</code> or a smaller positive block size; use <code>solve_adjoint</code> when a full trajectory or lower memory is more important than differentiating the discrete solver.
A full example is slow	Run without <code>PULSEPROP_FULL_EXAMPLES</code> first; the full grids and trajectory counts are intentionally computationally intensive.
CUDA.jl loads but no GPU is usable	Check <code>nvidia-smi</code> , pod GPU assignment, <code>CUDA_VISIBLE_DEVICES</code> , then run using <code>CUDA; CUDA.versioninfo(); CUDA.functional()</code> .
A CUDA method error	Use fixed-step <code>RK4IP()</code> or fixed-step <code>Tsit5()</code> for forward propagation and fixed-step <code>Vern9()</code> for the adjoint.
A full-field shape error	Retain <code>(time, x, y, polarization)</code> and use a singleton polarization axis; attach a uniform <code>TransverseGrid</code> to a <code>:time, :space</code> domain.
A full-field polarization or CPU error	The experimental full-field solver is scalar and CUDA-only; do not activate <code>:polarization</code> , and use <code>backend=:cuda</code> .
An RK4IP full-field stability error	Reduce <code>dz</code> until <code>min(dz,length)*maximum(abs,potential) <= 2sqrt(2)</code> , then continue reducing it to test accuracy convergence.
CPU/GPU validation misses tolerance	First reduce <code>dz</code> , confirm the same CP checksum and saved planes, keep <code>synchronize=true</code> , and compare observables as well as relative L2 error.
A RunPod result disappears	Store output and both cache files on a mounted persistent volume before terminating the pod.